



# Computer Vision OpenCV

Nada Fitrieyatul Hikmah

# Konsep Dasar Video

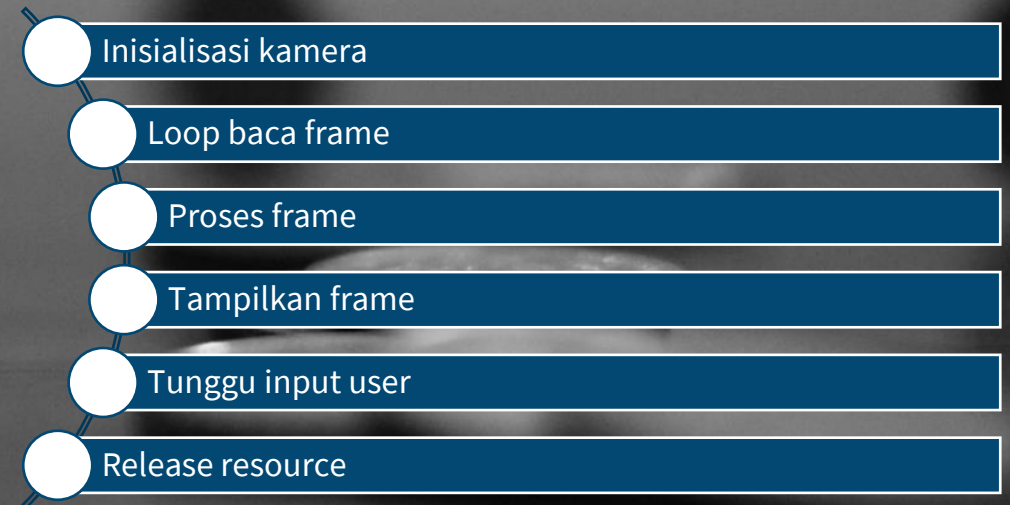
- Video = kumpulan frame (citra statis)
- Frame rate (fps) menentukan kelancaran video
- Setiap frame diproses seperti image biasa
- OpenCV membaca video sebagai stream

## OPENCV UNTUK VIDEO

Fungsi utama:

- `cv2.VideoCapture()` → akses kamera/video
- `cap.read()` → ambil frame
- `cv2.imshow()` → tampilkan frame
- `cv2.waitKey()` → kontrol input keyboard

## ALUR PROGRAM:



# Connecting to Camera (1) - grayscale

Pipeline real-time vision:

Capture (ambil data) → Process (ubah citra) →

Display (tampilkan) → Control (user bisa stop program)

```
import cv2

cap = cv2.VideoCapture(0)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

while True:

    ret, frame = cap.read()

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow('frame', gray)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Index kamera:

- 0 → webcam utama
- 1 → kamera eksternal (jika ada)

Mengambil ukuran frame video

- ret → True jika berhasil baca frame
- frame → citra (numpy array)

Konversi ke grayscale agar lebih ringan (1 channel)

- Tunggu 1 ms input keyboard & 0xFF → masking agar kompatibel semua OS
- ord('q') → ASCII huruf q

Menghentikan loop saat tombol 'q' ditekan

- Menutup koneksi kamera agar tidak “terkunci”
- Menutup semua GUI window OpenCV



# Connecting to Camera (2) - BGR

Pipeline real-time vision:

Capture (ambil data) → Process (ubah citra) → Display (tampilkan) → Control (user bisa stop program)

```
import cv2

cap = cv2.VideoCapture(0)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

while True:

    ret, frame = cap.read()

    #   gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow('frame', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

# Saving a video (OpenCV-VideoWriter)

Webcam → Capture Frame → Simpan ke File → Tampilkan → Loop

```
# https://docs.opencv.org/4.x/dd/d43/tutorial\_py\_video\_display.html
#Saving a video

import cv2

cap = cv2.VideoCapture(0)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

writer = cv2.VideoWriter('cobacoba.mp4',cv2.VideoWriter_fourcc(*'DIVX'),20,(width,height))

while True:

    ret, frame = cap.read()

    #operations (drawing)
    writer.write(frame)

    cv2.imshow('frame',frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
writer.release()
cv2.destroyAllWindows()
```

pwd

'C:\\Users\\Nada FH\\Computer Vision'

Format encoding video. DIVX = salah satu codec populer. Codec = Coder + DECoder → Cara mengubah (encode) video mentah menjadi file terkompresi, dan membacanya kembali (decode). Tanpa codec? Ukuran video bisa ratusan MB hanya dalam beberapa detik. Tidak semua jenis codec cocok di OS.

Code lain yg resolusinya besar → MJPG

-----  
20 FPS (Frame per second)  
-----

Setiap FPS nya HARUS sama dengan frame input  
-----

Simpan frame ke file video  
-----

Menutup proses penulisan video dan memastikan file tersimpan dengan benar  
-----

Print Working Directory  
Digunakan untuk: Menampilkan lokasi folder (directory) tempat kita sedang bekerja sekarang

# Using Video Files

Pipeline real-time vision:

Video File → VideoCapture → Frame Extraction (cap.read) → Display → Timing control (FPS) → user stop

```
import cv2
import time

cap = cv2.VideoCapture('cobacoba.mp4')

if cap.isOpened() == False:
    print('file error atau codec salah')

while cap.isOpened():

    ret, frame = cap.read()

    if ret == True:

        #Writer 20 FPS:
        time.sleep(1/20)
        cv2.imshow('frame', frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    else:
        break

cap.release()
cv2.destroyAllWindows()
```

Ambil 1 frame pada satu waktu.  
Output: frame (citra), ret (status)

---

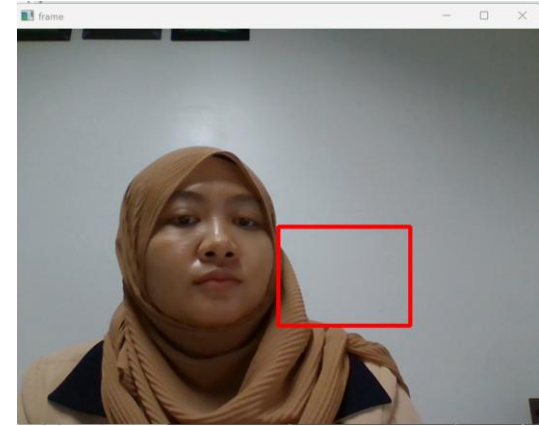
Delay agar playback sesuai **20 FPS**  
Tanpa kodingan ini:  
Video terlalu cepat

Program 'pause' selama 1/20 detik = 0.05detik →  
agar video menjadi 20 FPS

FPS terlalu kecil → video lambat  
FPS terlalu besar → video cepat

---

# Drawing on Live Camera (manual)



Pipeline :

Webcam → Capture Frame → Tentukan ROI (Rectangle) → Gambar Rectangle → Display → Loop.

Basic pipeline untuk ROI selection, Face detection, Tracking, rPPG (ambil area wajah)

```
import cv2

cap = cv2.VideoCapture(0)

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

#Top Left Corner
x = width // 2
y = height // 2

#width n height of RECTANGLE
w = width // 4
h = height // 4

#BOTTOM RIGHT ==> x + w, y + h

while True:

    ret, frame = cap.read()

    cv2.rectangle(frame, (x,y),(x+w,y+h),color=(0,0,255),thickness=4)

    cv2.imshow('frame',frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Titik (x, y) = tengah frame

Lebar & tinggi rectangle = 1/4 ukuran frame

Jadi, konsep koordinat:

Top-left : (x, y)

Bottom-right : (x + w, y + h)

(x,y) → titik kiri atas

(x+w, y+h) → kanan bawah

(0,0,255) → warna merah (**BGR**)

thickness=4 → ketebalan garis

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
#CALLBACK FUNCTION RECTANGLE
```

```
def draw_rectangle(event,x,y,flags,param):
```

```
    global pt1,pt2,topLeft_clicked,botRight_clicked
```

```
    if event == cv2.EVENT_LBUTTONDOWN:
```

```
        #RESET THE RECTANGLE (cek apa ada rect disana)
```

```
        if topLeft_clicked == True and botRight_clicked == True:
```

```
            topLeft_clicked = False
```

```
            botRight_clicked = False
```

```
            pt1 = (0,0)
```

```
            pt2 = (0,0)
```

```
        if topLeft_clicked == False:
```

```
            pt1 = (x,y)
```

```
            topLeft_clicked = True
```

```
        elif botRight_clicked == False:
```

```
            pt2 = (x,y)
```

```
            botRight_clicked = True
```

```
#GLOBAL VARIABLES
```

```
pt1 = (0,0)
```

```
pt2 = (0,0)
```

```
topLeft_clicked = False
```

```
botRight_clicked = False
```

```
#CONNECT TO THE CALLBACK
```

```
cap = cv2.VideoCapture(0)
```

```
cv2.namedWindow('Test')
```

```
cv2.setMouseCallback('Test',draw_rectangle)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    #Drawing on the frame based off the global variables
```

```
    if topLeft_clicked:
```

```
        cv2.circle(frame,center=pt1,radius=5,color=(0,0,255),thickness=-1)
```

```
    if topLeft_clicked and botRight_clicked:
```

```
        cv2.rectangle(frame,pt1,pt2,(0,0,255),3)
```

```
    cv2.imshow('Test',frame)
```

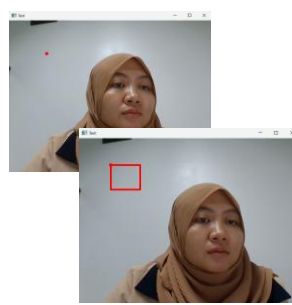
```
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

# Drawing on Live Camera (live drawing)



Pipeline :

Webcam → Display Frame → Mouse Click (User Input) → Simpan Koordinat (pt1, pt2) → Gambar Titik & Rectangle.

Basic pipeline untuk ROI selection, Face detection, Tracking, rPPG (ambil area wajah)

Event saat klik kiri mouse

Jika sudah ada rectangle:  
reset semua dan mulai ulang

Simpan titik awal rectangle

Simpan titik akhir rectangle

Titik awal, akhir rectangle, dan status kliknya

Menghubungkan window "Test" dengan fungsi mouse

Tampilkan titik merah saat klik pertama, thickness -1 =  
filled circle

Gambar rectangle jika dua titik sudah ada



# Mini Assignment

---

Create a program that reads in a live stream from a camera on your computer. Then whenever you click the left mouse button down, create a blue circle around where you've clicked.

A low-angle, black and white photograph of several tall skyscrapers reaching towards a bright sky. The perspective creates a sense of height and scale. In the lower-left foreground, the back of a person's head and shoulders are visible, looking up at the buildings. A solid teal horizontal bar spans the width of the image, partially obscured by a dark grey rectangular box containing the title text.

# OBJECT DETECTION WITH OPENCV

# METHODS (1)

- ❑ Template Matching
  - ❑ Simply looking for an exact copy of an image in another image
- ❑ Corner Detection
  - ❑ Looking for corners in images
- ❑ Edge Detection
  - ❑ Expanding to find general edges of objects
- ❑ Grid Detection
  - ❑ Combining both concepts to find grids in images (useful for applications)
- ❑ Contour Detection
  - ❑ Allows us to detect foreground vs background images
  - ❑ Also allows for detection of external vs internal contours (e.g. grabbing the eyes and smile from a cartoon smile face)

# METHODS (2)

## ☐ Feature Matching

- ☐ More advanced methods of detecting matching objects in another image, even if the target image is not shown exactly the same in the image we are searching

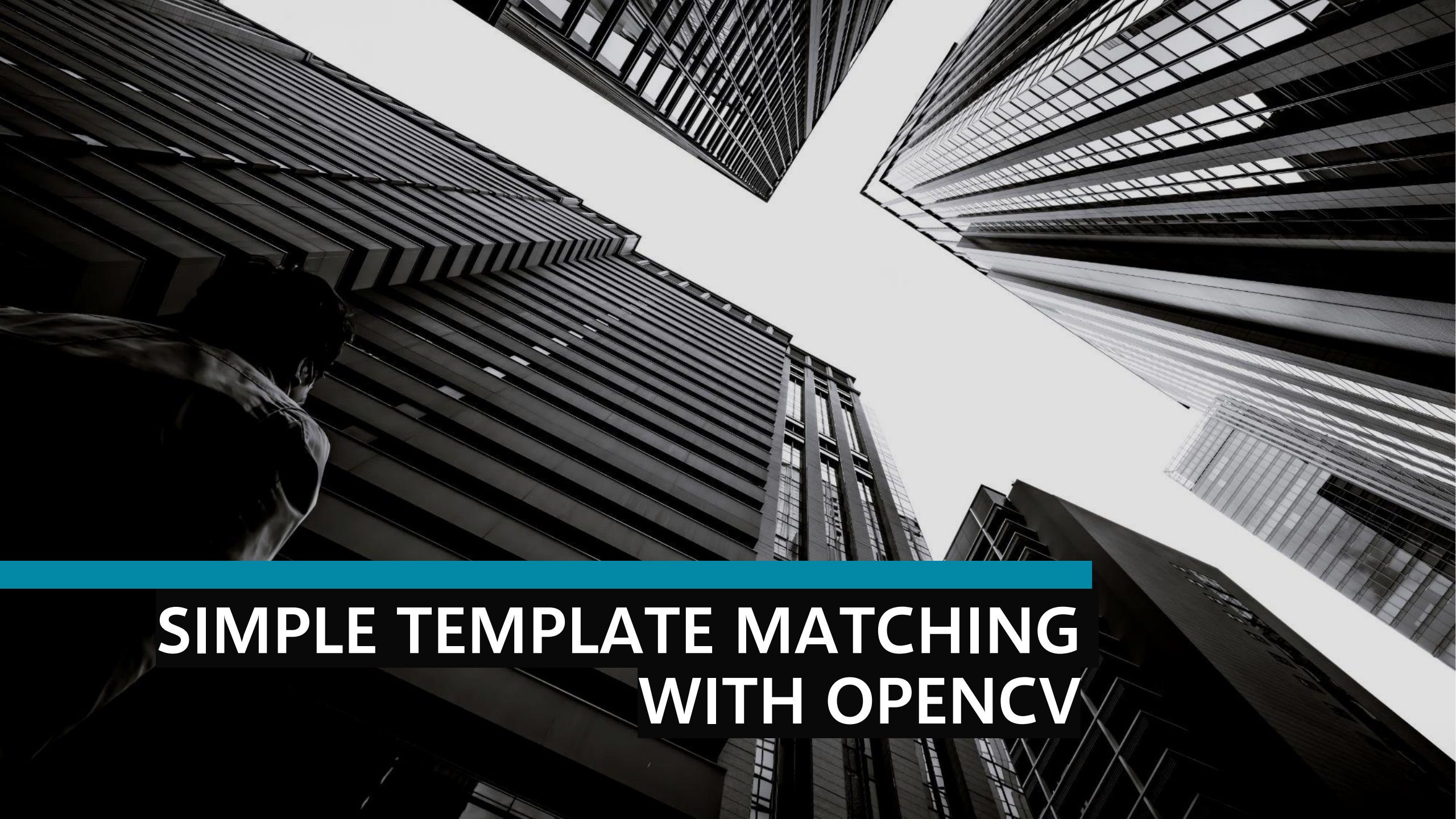
## ☐ Watershed Algorithm

- ☐ Advanced algorithm that allows us to segment images into foreground and background
- ☐ Allow us to manually set seeds to choose segments of an image

## ☐ Facial and Eye Detection

- ☐ Haar Cascades to detect faces in images
- ☐ Require deep learning for high accuracy



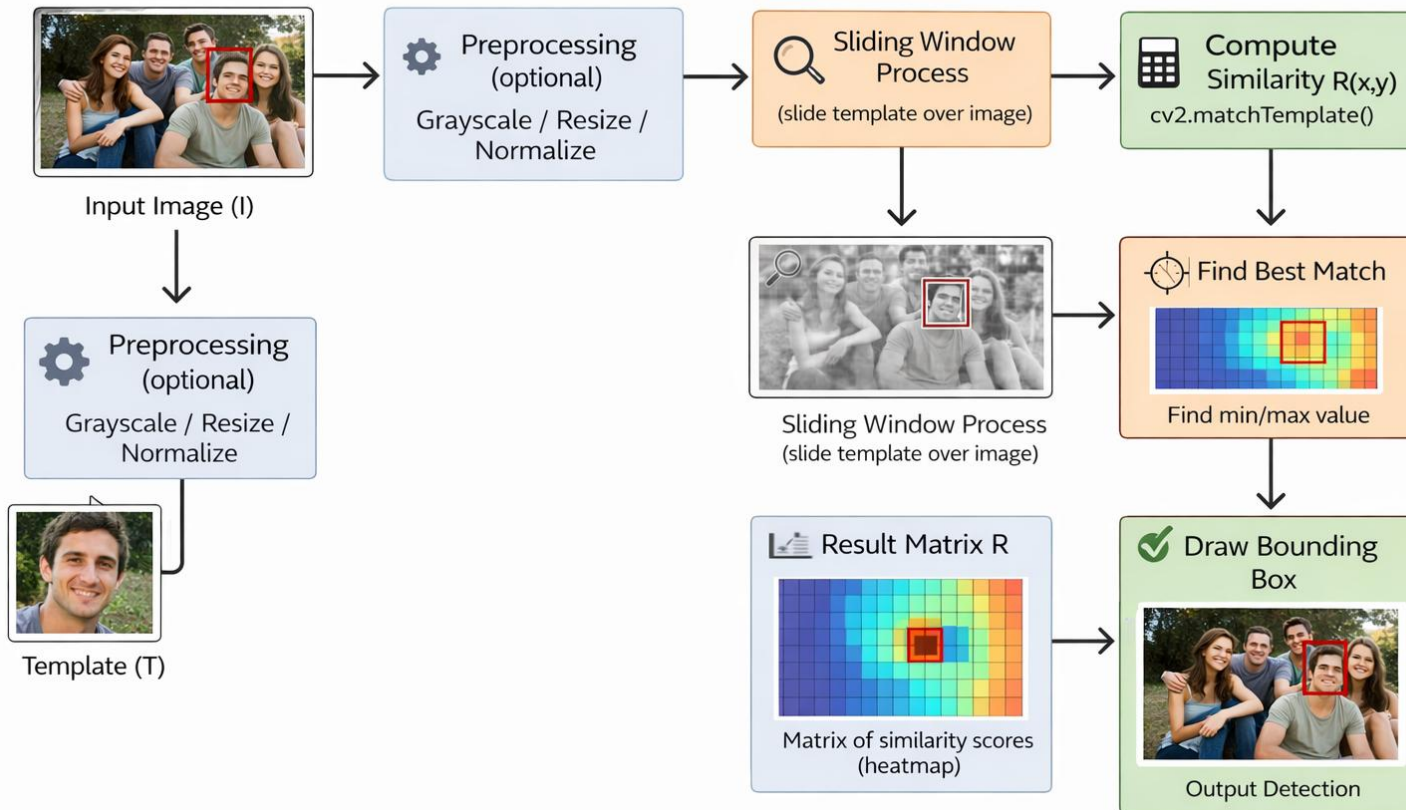


# **SIMPLE TEMPLATE MATCHING WITH OPENCV**

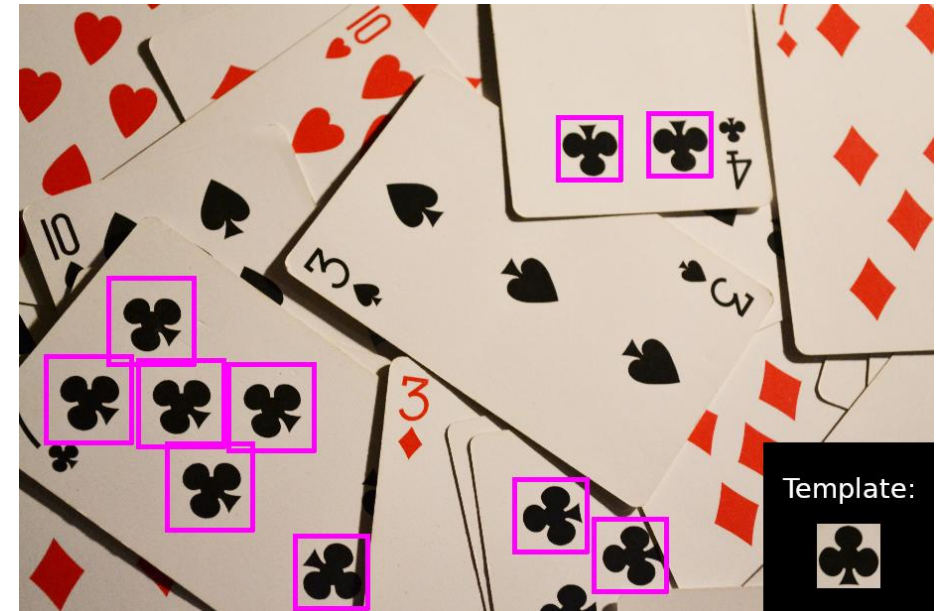


# TEMPLATE MATCHING (1)

## Template Matching Pipeline



- ❑ Template Matching adalah metode paling sederhana untuk object detection
- ❑ IDE utama: Mencari bagian gambar yang paling mirip dengan template (contoh objek).



# Load Image and Template

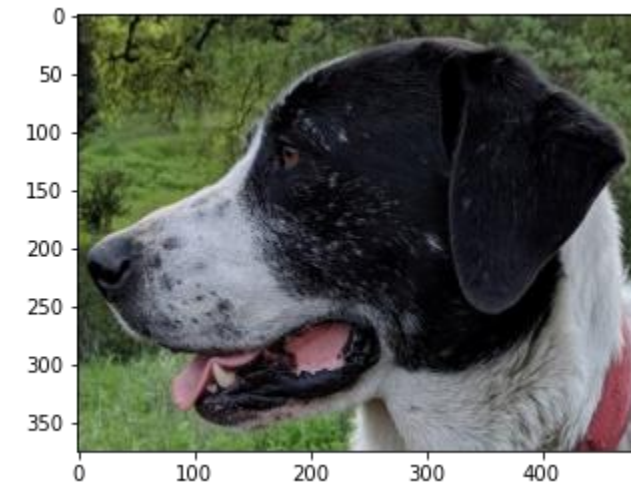
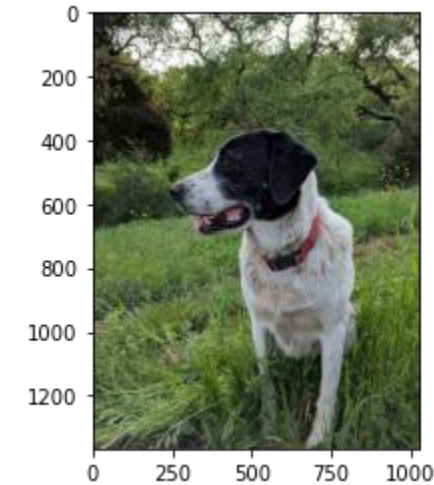
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

```
full = cv2.imread('dog.jpg')
full = cv2.cvtColor(full, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(full)
```

```
face = cv2.imread('dog_face.jpg')
face = cv2.cvtColor(face, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(face)
```



# Template Size & Methods

- ❑ Digunakan untuk bounding box nanti.

```
height, width, channels = face.shape
```

```
height
```

```
375
```

```
width
```

```
486
```

| Enumerator                                      |                                                                                                                                                                                                                                                            |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TM_SQDIFF<br>Python: cv.TM_SQDIFF               | $R(x, y) = \sum_{x', y'} (T(x', y') - I(x + x', y + y'))^2$                                                                                                                                                                                                |
| TM_SQDIFF_NORMED<br>Python: cv.TM_SQDIFF_NORMED | $R(x, y) = \frac{\sum_{x', y'} (T(x', y') - I(x + x', y + y'))^2}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$                                                                                                               |
| TM_CCORR<br>Python: cv.TM_CCORR                 | $R(x, y) = \sum_{x', y'} (T(x', y') \cdot I(x + x', y + y'))$                                                                                                                                                                                              |
| TM_CCORR_NORMED<br>Python: cv.TM_CCORR_NORMED   | $R(x, y) = \frac{\sum_{x', y'} (T(x', y') \cdot I(x + x', y + y'))}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$                                                                                                             |
| TM_CCOEFF<br>Python: cv.TM_CCOEFF               | $R(x, y) = \sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))$ <p>where</p> $T'(x', y') = T(x', y') - 1/(w \cdot h) \cdot \sum_{x'', y''} T(x'', y'')$ $I'(x + x', y + y') = I(x + x', y + y') - 1/(w \cdot h) \cdot \sum_{x'', y''} I(x + x'', y + y'')$ |
| TM_CCOEFF_NORMED<br>Python: cv.TM_CCOEFF_NORMED | $R(x, y) = \frac{\sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))}{\sqrt{\sum_{x', y'} T'(x', y')^2 \cdot \sum_{x', y'} I'(x + x', y + y')^2}}$                                                                                                         |

# Template Matching – Correlation Coefficient (TM\_CCOEFF)

$$R(x, y) = \sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))$$

$$T' = T - \bar{T}, \quad I' = I - \bar{I}$$

- ❑  $T(x', y')$  = TEMPLATE → gambar kecil (objek yang dicari)
- ❑  $I(x, y)$  = IMAGE BESAR (CITRA ASLI)
- ❑  $(x, y)$  = POSISI TEMPLATE DI IMAGE
- ❑  $(x', y')$  = KOORDINAT DI DALAM TEMPLATE

STEP:

1. Geser template ke posisi  $(x, y)$
2. Ambil patch image:  $I(x + x', y + y')$
3. Kurangi mean:  $T'$  dan  $I'$
4. Kalikan pixel:  $T' * I'$
5. Jumlahkan semua:  $\sum$  semua pixel

```
# all 6 methods TM
methods = ['cv2.TM_CCOEFF', 'cv2.TM_CCOEFF_NORMED', 'cv2.TM_CCORR', 'cv2.TM_

for m in methods:

    # dicopy agar image asli tdk berubah
    full_copy = full.copy()

    # Ambil method dari string
    method = eval(m)

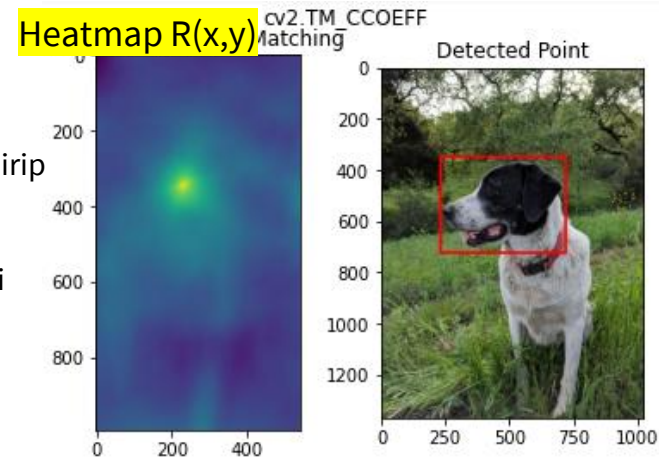
    # Apply TM with the method
    res = cv2.matchTemplate(full_copy, face, method)

my_method = eval('cv2.TM_CCOEFF')
res = cv2.matchTemplate(full_copy, face, my_method)

plt.imshow(res)
```

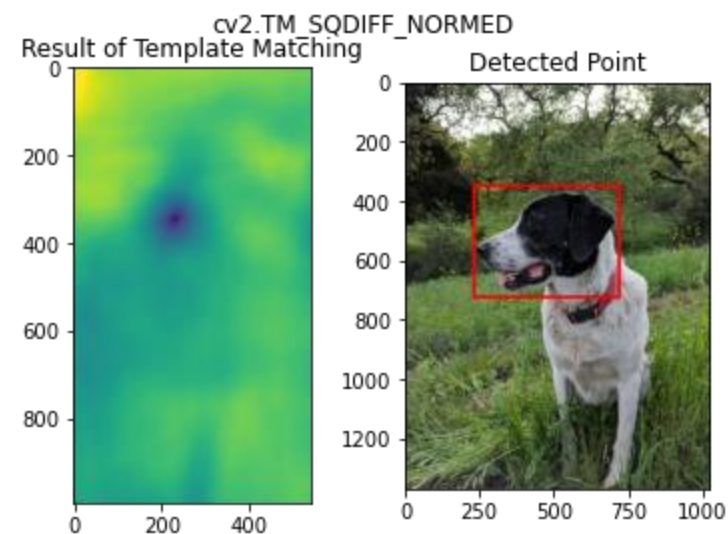
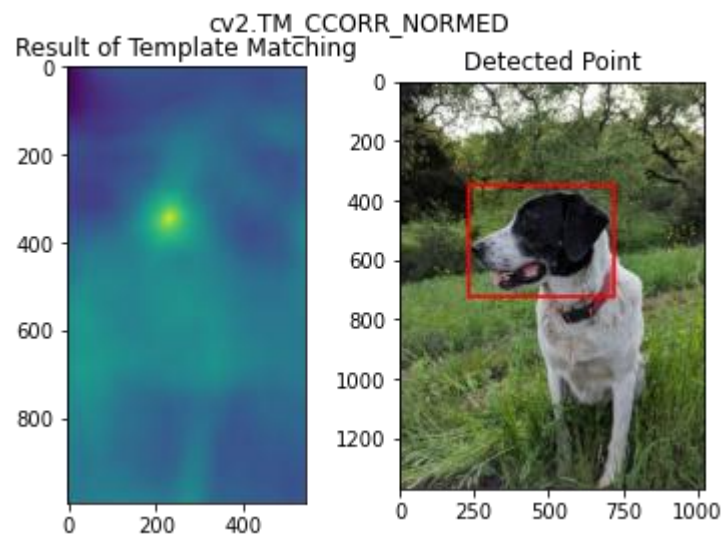
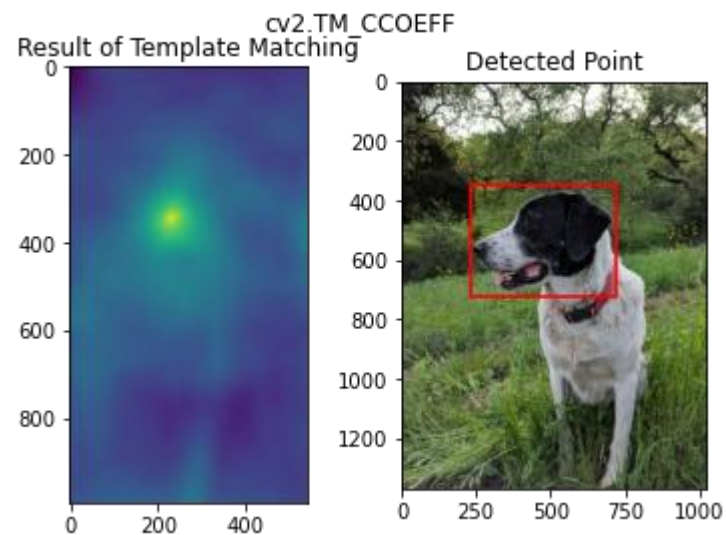
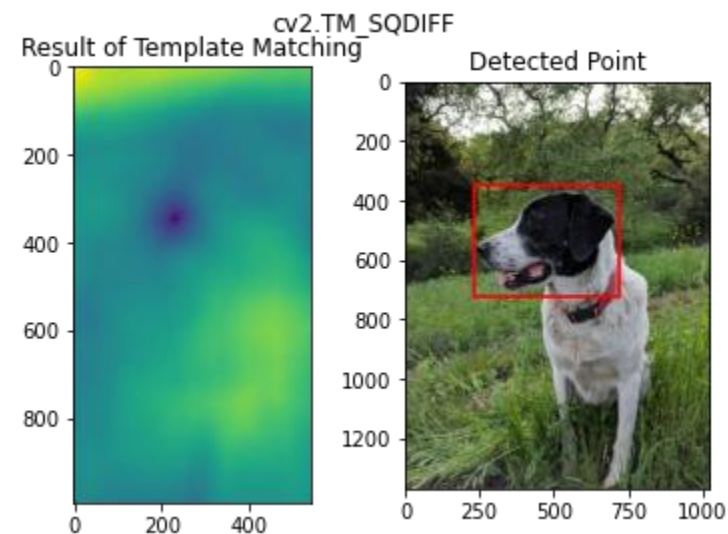
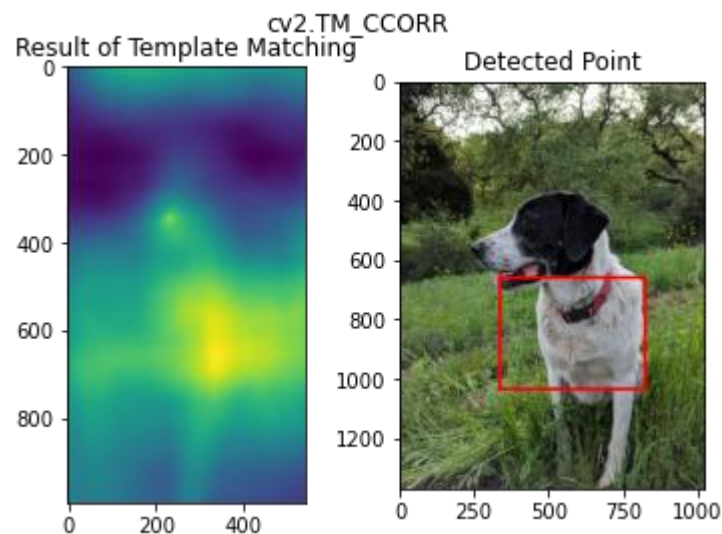
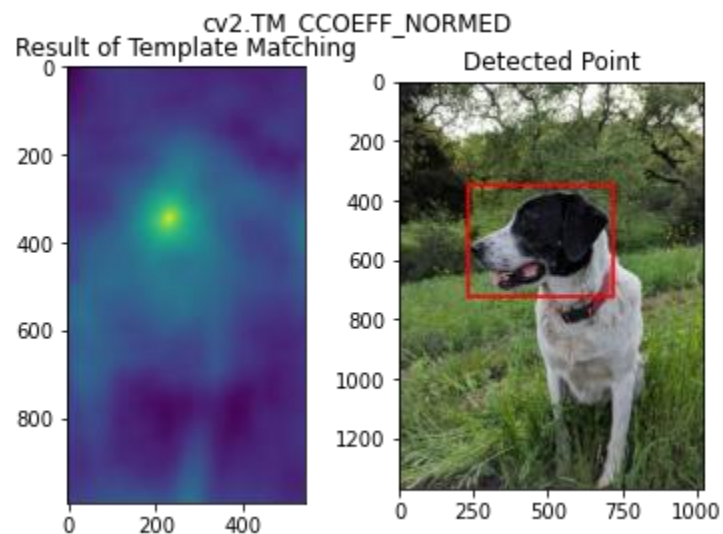
terang → nilai tinggi → mirip  
gelap → tidak mirip

TM\_CCOEFF → cari nilai  
**MAXIMUM**





# Results



# Comparison of TM Methods

| Metode                  | Prinsip                    | Rumus Inti             | Ambil Nilai | Kelebihan            | Kekurangan                    |
|-------------------------|----------------------------|------------------------|-------------|----------------------|-------------------------------|
| <b>TM_SQDIFF</b>        | Selisih pixel              | $(\sum (T - I)^2)$     | <b>MIN</b>  | Sederhana            | Sensitif brightness           |
| <b>TM_SQDIFF_NORMED</b> | Selisih + normalisasi      | normalized SSD         | <b>MIN</b>  | Lebih stabil         | Masih sensitif noise          |
| <b>TM_CCORR</b>         | Perkalian langsung         | $(\sum T \cdot I)$     | <b>MAX</b>  | Cepat                | Tidak robust lighting         |
| <b>TM_CCORR_NORMED</b>  | Correlation + normalisasi  | normalized dot product | <b>MAX</b>  | Lebih stabil         | Masih bias intensitas         |
| <b>TM_CCORR_NORMED</b>  | Correlation (mean removed) | $(\sum T' \cdot I')$   | <b>MAX</b>  | Tahan brightness     | Tidak scale invariant         |
| <b>TM_CCORR_NORMED</b>  | Correlation + normalisasi  | normalized correlation | <b>MAX</b>  | <b>Paling robust</b> | Sedikit lebih mahal komputasi |



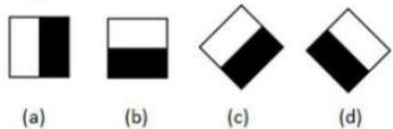
A black and white photograph taken from a low angle looking up at several tall, modern skyscrapers. The buildings are made of glass and steel, with many windows reflecting light. In the bottom left foreground, the back of a person's head and shoulders are visible, looking towards the buildings. A solid blue horizontal bar is positioned across the middle of the image, just above the text.

# FACE DETECTION WITH OPENCV

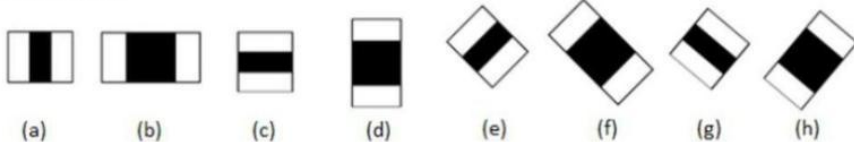
# Haar Cascade

- ❑ Metode klasik untuk deteksi objek (terutama wajah) menggunakan fitur sederhana berbasis perbedaan intensitas.
- ❑ Deteksi wajah dengan pola terang-gelap sederhana (Haar-like features)
- ❑ Haar-like Features: Feature = (jumlah pixel area putih) – (jumlah pixel area hitam)

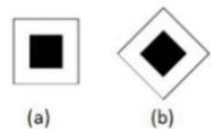
(1) Edge features



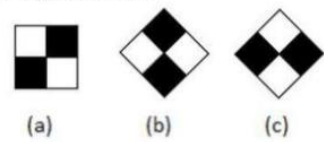
(2) Line features



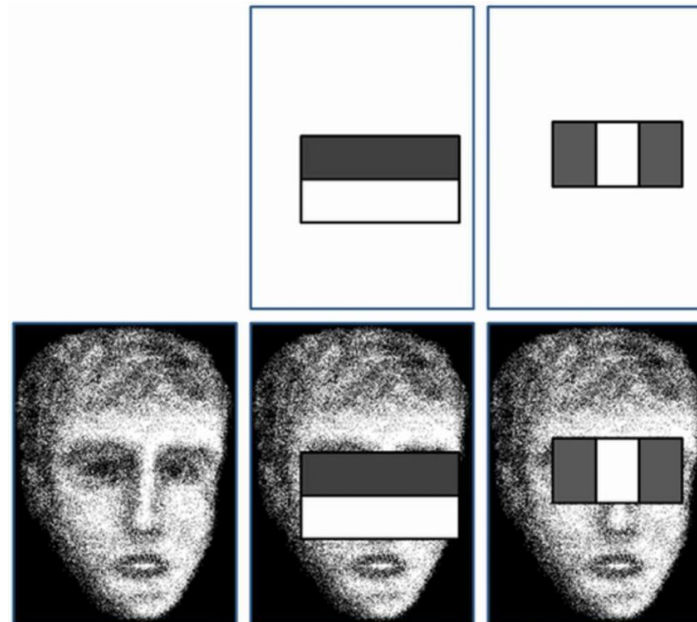
(3) Center-surround features



(4) Diagonal features



1. **Two-rectangle feature**  
Deteksi perbedaan terang-gelap  
Contoh: mata (gelap) vs pipi (terang)
2. **Three-rectangle feature**  
Deteksi garis (line)  
Contoh: hidung
3. **Four-rectangle feature**  
Deteksi perubahan diagonal  
Contoh: tekstur kompleks



Wajah manusia mempunyai pola konsisten:

| Area   | Karakteristik |
|--------|---------------|
| Mata   | lebih gelap   |
| Pipi   | lebih terang  |
| Hidung | garis tengah  |



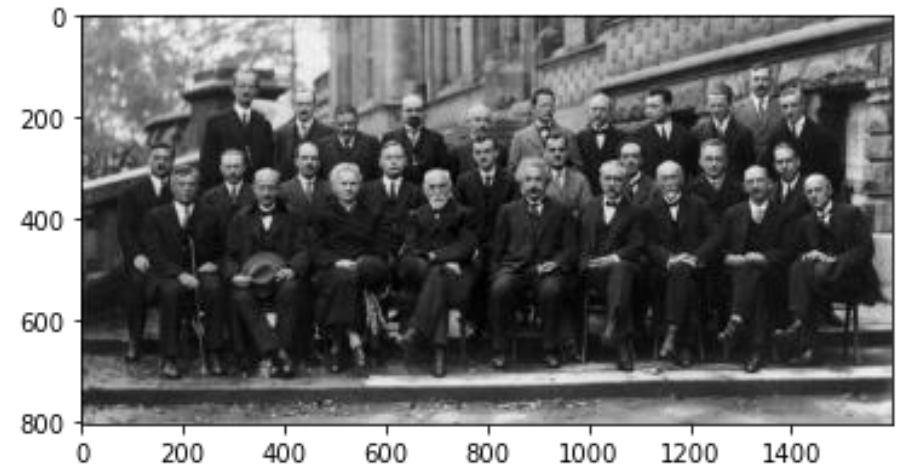
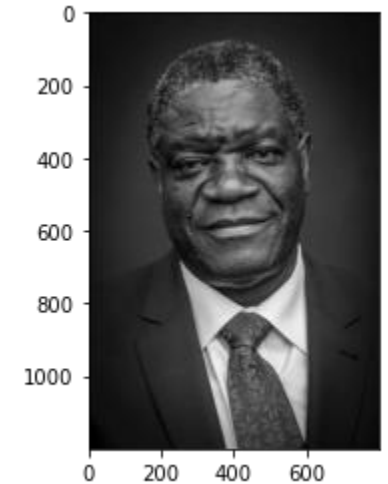
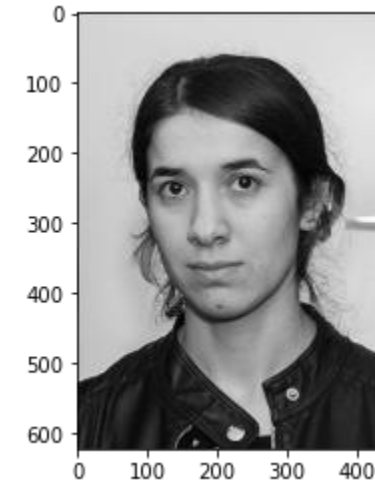
# Load Image

## 1 Face Detection with Haar Cascades

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
%matplotlib inline
```

### 1.1 Images

```
nadia = cv2.imread('Nadia_Murad.jpg',0) # 0 = flag for grey colour
denis = cv2.imread('Denis_Mukwege.jpg',0)
solway = cv2.imread('solway_conference.jpg',0)
```



# LOAD HAAR CASCADE

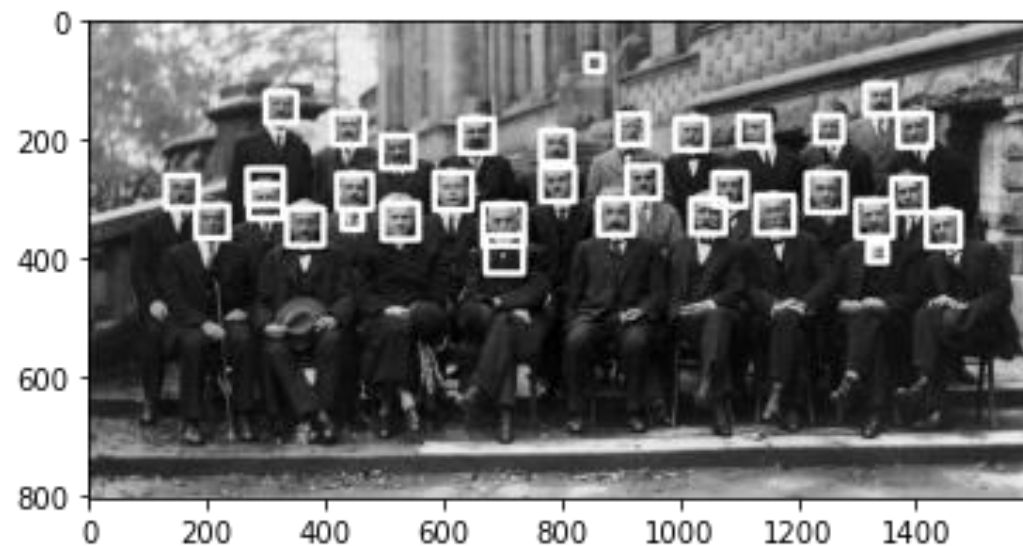
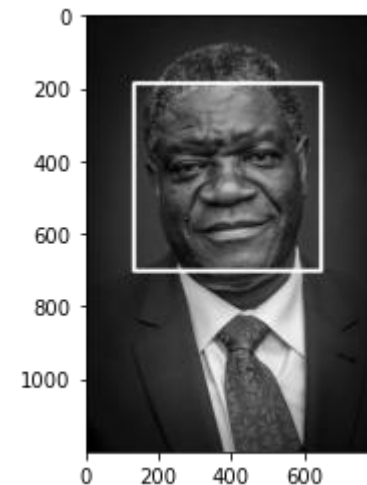
```
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

- ❑ Buka: <https://github.com/opencv/opencv/tree/master/data/haarcascades>
- ❑ Load model pre-trained .xml yg berisi: Haar features (hasil training AdaBoost)
- ❑ XML = eXtensible Markup Language → karena Haar cascade berbasis rule-based
- ❑ detectMultiScale(): Sliding window → Haar feature extraction → Cascade classifier → facedetection

```
def detect_face(img):  
  
    face_img = img.copy()  
  
    face_rects = face_cascade.detectMultiScale(face_img)  
  
    for (x,y,w,h) in face_rects:  
        cv2.rectangle(face_img, (x,y), (x+w,y+h), (255,255,255), 10)  
  
    return face_img
```

```
result = detect_face(denis)
```

```
plt.imshow(result,cmap='gray')
```



# Adjustment Parameter

```
#coba diperbaiki
def adj_detect_face(img):

    face_img = img.copy()

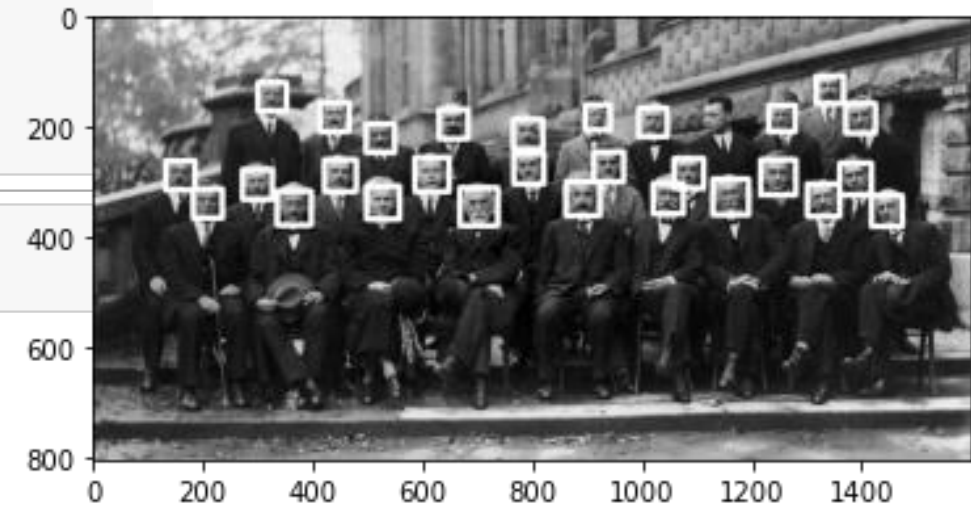
    face_rects = face_cascade.detectMultiScale(face_img,scaleFactor=1.2, minNeighbors=5)

    for (x,y,w,h) in face_rects:
        cv2.rectangle(face_img, (x,y), (x+w,y+h), (255,255,255), 10)

    return face_img

result = adj_detect_face(solvay)
plt.imshow(result,cmap='gray')
```

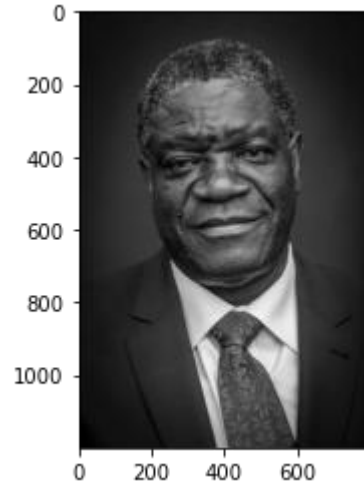
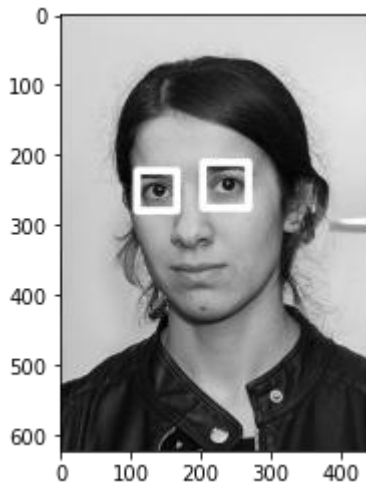
- ❑ minNeighbors = 5 → Berapa banyak "deteksi tetangga" yang harus saling mendukung
  - ❑ hanya 1-2 kotak → kemungkinan noise,
  - ❑ banyak kotak bertumpuk → kemungkinan wajah
- ❑ scaleFactor = 1.2 → Seberapa cepat ukuran window berubah
  - ❑ Window = kotak kecil (region) yang dipakai untuk mengecek apakah di situ ada wajah
  - ❑ Jadi ukuran window diperbesar 1.2x tiap Langkah → utk cek wajah posisi dekat kamera (besar) dan jauh kamera (kecil)



# Eye Cascade File

```
eye_cascade = cv2.CascadeClassifier('haarcascade_eye.xml')
```

```
def detect_eyes(img):  
    face_img = img.copy()  
  
    eyes = eye_cascade.detectMultiScale(face_img)  
  
    |  
    for (x,y,w,h) in eyes:  
        cv2.rectangle(face_img, (x,y), (x+w,y+h), (255,255,255), 10)  
  
    return face_img
```



Tidak terdeteksi?

- ☐ Kontras mata kurang jelas
- ☐ Tidak menggunakan ROI wajah



# Live Video

```
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read(0)

    frame = detect_face(frame)

    cv2.imshow('Video Face Detection', frame)

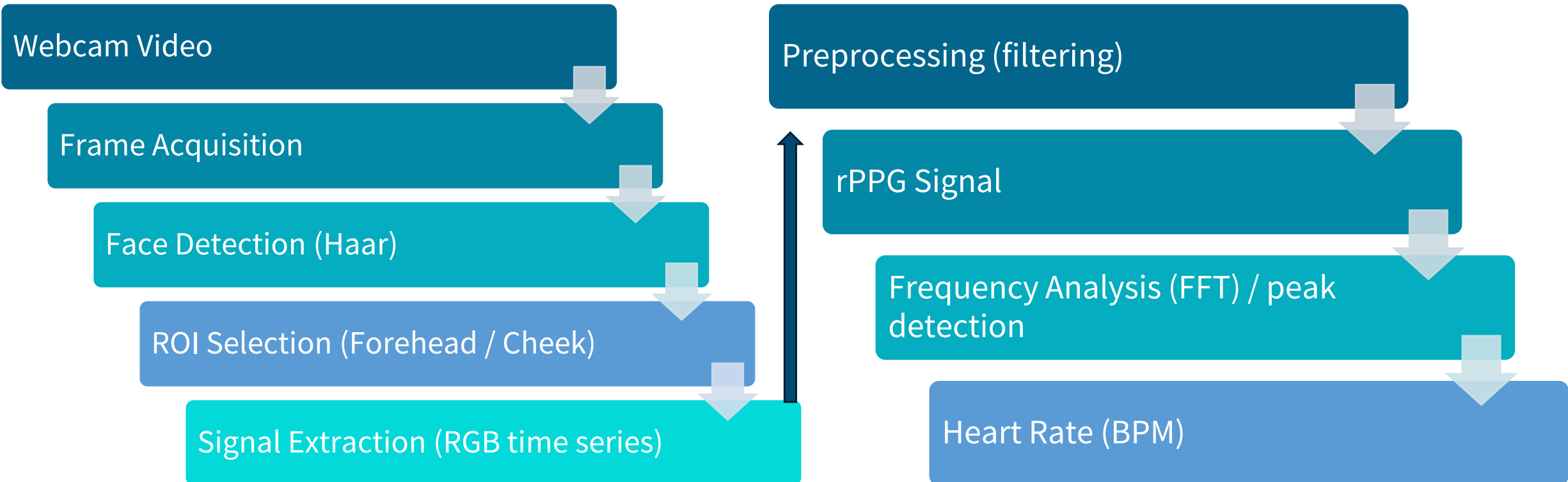
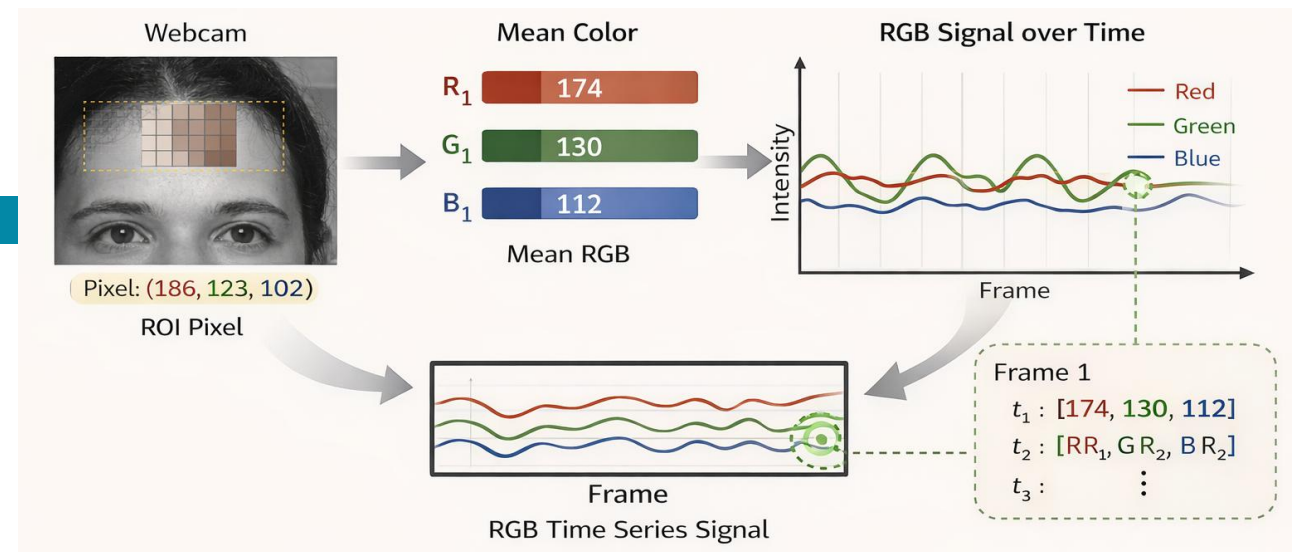
    c = cv2.waitKey(1)
    if c == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

- ❑ Webcam → ambil frame (video) → detect\_face → gambar bounding box → tampilkan
- ❑ cv2.waitKey(1) → Tunggu 1 ms untuk input keyboard
- ❑ c == 27 → misalnya Esc utk quit (27 = ASCII dari Escape)

# Assignment

- ❑ Buat program live webcam untuk deteksi wajah, deteksi mata, dan estimasi nilai HR.





**THANK YOU**